

SOFTWARE REQUIREMENTS SPECIFICATION

Adaptive Explainable Multi-Agent Retrieval-Augmented Framework for Intelligent Document Analysis

Version 1.0

Major Project / Research-Oriented Software System

Document Item	Details
Institution	Keshav Memorial Institute of Technology
Project Type	Major Project / Research-Oriented Software System
Domain	RAG, Multi-Agent AI, Intelligent Document Analysis
Frontend	Next.js, TypeScript, Tailwind CSS
Backend	Python FastAPI
AI Orchestration	LangGraph / LangChain
Retrieval	BM25 + Dense Retrieval (Qdrant) + Hybrid Retrieval
Embeddings	BAAI BGE family embedding model
Database	PostgreSQL + Qdrant
LLM	Configurable Gemini / OpenAI-compatible provider
Supported Documents	PDF, DOCX, TXT, reports, manuals, research papers
Date	September 2026

Prepared using the structure of the supplied Startup Idea Validator SRS Version 4.0. That reference document is an 11-page Keshav Memorial Institute of Technology SRS and contains the same broad sections used here: introduction, overall description, interfaces, system features, nonfunctional requirements and appendices. ■filecite■turn17file0■L1-L8■

. Table of Contents

Section	Title
1	Introduction
2	Overall Description
3	External Interface Requirements
4	System Features and Functional Requirements
5	Adaptive Retrieval and AI Processing Requirements
6	Multi-Agent, Verification and Explainability Requirements
7	Data Requirements and API Requirements
8	System Architecture and Component Requirements
9	Other Nonfunctional Requirements
10	Evaluation and Research Requirements
11	Testing and Validation Requirements
12	Business Rules and Operational Policies
13	Deployment and Infrastructure Requirements
14	Future Enhancements and TBD Items
Appendix A	Glossary
Appendix B	Analysis Models
Appendix C	Requirement Traceability Matrix
Appendix D	End-to-End Implementation Checklist
Appendix E	Research Dataset Schema

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for an adaptive, explainable, multi-agent Retrieval-Augmented Generation (RAG) framework for intelligent document analysis. The system allows users to upload documents, ask natural-language questions, request summaries or comparisons, and receive evidence-backed responses with source references, supporting passages, verification status and confidence information.

The product is also a research platform. It must expose measurable retrieval, reranking and verification behavior so that adaptive retrieval can be compared with fixed retrieval baselines under a controlled experimental protocol.

1.2 Document Conventions

- Functional requirements use REQ-Fxx; non-functional requirements use REQ-NFxx; security requirements use REQ-Sxx; data requirements use REQ-Dxx; evaluation requirements use REQ-EVxx.
- 'Shall' means mandatory; 'Should' means recommended; 'May' means optional.
- Priority levels are High, Medium and Low.

1.3 Intended Audience and Reading Suggestions

This document is intended for project developers, testers, faculty reviewers, research evaluators, administrators and future maintainers. Sections 1–3 explain scope and interfaces. Sections 4–8 define functional, AI/RAG, data and architecture requirements. Sections 9–11 define quality and validation requirements. Appendices provide terminology, analysis models, traceability and implementation checklists.

1.4 Product Scope

The system provides a controlled document-intelligence workspace. Documents are parsed into retrievable chunks, indexed lexically and semantically, retrieved using an adaptive strategy, reranked, processed through a bounded multi-agent workflow, verified against evidence, and presented with explainability information.

The system is not specified as a replacement for general-purpose conversational AI. Its distinguishing research focus is explicit adaptive retrieval, evidence verification, provenance, confidence-aware explanation and measurable evaluation.

1.5 References

- Supplied Startup Idea Validator SRS Version 4.0, Keshav Memorial Institute of Technology, used as the structural reference. Its table of contents and feature organization are visible in the supplied document.
■filecite■turn17file0■L11-L38■
- Project abstract/proposal covering adaptive retrieval, LangGraph multi-agent processing, evidence verification and explainability.
- Project architecture and implementation design maintained by the project team.

2. Overall Description

2.1 Product Perspective

The system is a full-stack web application consisting of a Next.js client, FastAPI application services, PostgreSQL transactional storage, Qdrant dense-vector storage, a BM25 lexical index and a LangGraph-based AI orchestration layer. The LLM is accessed through a configurable provider adapter.

The separation of ingestion, retrieval, reasoning, verification and presentation is intentional: each component can be independently tested, replaced and evaluated.

2.2 High-Level Product Functions

- Registration, login, logout, token refresh and password management.
- Document upload, validation, processing status and lifecycle management.
- PDF, DOCX and TXT parsing with text normalization and provenance metadata.
- Configurable structural/semantic-aware chunking.
- BGE dense embeddings and Qdrant vector storage.
- BM25 lexical indexing for exact-term retrieval.
- Dense, BM25 and Hybrid retrieval.
- Adaptive Retrieval Selection using query characteristics/diagnostics.
- Cross-Encoder reranking.
- LangGraph orchestration of supervisor, retrieval, verification and synthesis responsibilities.
- Question answering, summarization, comparison and citation/evidence tasks.
- Evidence verification with claim-to-source mapping and corrective retrieval.
- Confidence-aware explainability with sources, passages and verification status.
- Query/conversation history and research logging.
- Baseline evaluation, ablation studies, metric export and visualization.
- Administrative monitoring of users, jobs, errors and service health.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Activities
General User / Student	Non-expert user needing document analysis	Upload, query, summarize, compare, inspect evidence
Researcher / Analyst	Needs traceability and measurable behavior	Inspect retrieval, run experiments, compare metrics
Administrator	Operates and governs the platform	Manage users, jobs, failures, health and audit data
Developer / Maintainer	Builds and deploys the system	Configure, test, debug and extend components

2.4 Operating Environment

- Modern Chrome, Edge or Firefox browser.
- Next.js + TypeScript frontend.
- Python 3.11+ FastAPI backend.
- PostgreSQL and Qdrant.
- BGE embeddings, Cross-Encoder reranker and configurable LLM.
- Docker/Docker Compose for local and demonstration deployment.

2.5 Design and Implementation Constraints

- User documents must remain isolated by authenticated user/collection identifiers.
- Retrieval mode selection must be recorded for evaluation.

- Document-grounded answers must be evidence-checked before being represented as verified.
- Experimental values must never be hard-coded as research results.
- Secrets must be supplied through environment/secret configuration.
- Long-running ingestion/evaluation should be asynchronous.

2.6 User Documentation

- In-app onboarding.
- File type/limitation help.
- Research guide for retrieval and metrics.
- Developer/deployment guide.

2.7 Assumptions and Dependencies

- Configured LLM provider is reachable and has quota.
- PostgreSQL and Qdrant are available.
- Documents contain extractable text unless OCR is added.
- Embedding/reranker versions are compatible with indexes.
- Users have internet access.

3. External Interface Requirements

3.1 User Interfaces

Screen	Purpose	Key Elements
Login / Register	Authentication	Credentials, validation, recovery
Dashboard	Workspace overview	Documents, recent queries, processing states
Document Upload	Add source material	Drag/drop, picker, validation, progress
Document Viewer	Inspect source	Metadata, page/section navigation, extracted text
Query Workspace	Ask questions	Prompt, document scope, task controls
Answer View	Show result	Answer, confidence, verification, citations, evidence
Retrieval Inspector	Research/debug	Selected mode, candidates, scores, reranking
Compare / Summarize	Structured analysis	Document selection, task type, output
History	Review interactions	Queries, answers, timestamps, source sets
Evaluation Dashboard	Research analysis	Baselines, metrics, latency, ablations
Admin Console	Operations	Users, jobs, health, audit

3.2 Hardware Interfaces

No specialized client hardware is required. Standard laptops, desktops, tablets and smartphones are supported. Server-side inference may require additional CPU/GPU resources depending on model deployment.

3.3 Software Interfaces

Interface	Requirement
Next.js ↔ FastAPI	HTTPS REST APIs with JSON payloads and authenticated requests.
FastAPI ↔ PostgreSQL	SQLAlchemy persistence for users, documents, queries, jobs, conversations, evaluation and audit records.
FastAPI ↔ Qdrant	Vector upsert/search with ownership and document metadata filters.
FastAPI ↔ BM25	Lexical index creation, update and top-K retrieval.
FastAPI ↔ Embeddings	Batch BGE inference with fixed configured dimension.
FastAPI ↔ Reranker	Query–chunk scoring and top-M selection.
FastAPI ↔ LLM	Configurable provider adapter with timeout, retry and failure handling.
LangGraph ↔ Tools	Typed retrieval, verification, document and synthesis tools.

3.4 Communications Interfaces

- Production traffic shall use HTTPS.
- Credentials shall not be transmitted in URLs.
- File upload shall use multipart/form-data or an approved secure upload mechanism.
- Long-running status may use polling, SSE or WebSocket.

4.1. User Registration and Authentication

Priority: High. Stimulus/response: submit credentials → validate → authenticate → issue credentials → authorize resources.

ID	Requirement	Priority
REQ-F01	Allow new users to register with a unique identity.	High
REQ-F02	Validate required fields and reject duplicate identities.	High
REQ-F03	Store passwords only as secure one-way hashes.	High
REQ-F04	Issue authenticated access credentials after successful login.	High
REQ-F05	Enforce token expiry and refresh policy.	High
REQ-F06	Provide logout/session invalidation.	Medium
REQ-F07	Provide password recovery if enabled.	Medium
REQ-F08	Enforce RBAC for user and administrator operations.	High

4.2. Document Upload and Management

Priority: High. Documents are the primary knowledge source.

ID	Requirement	Priority
REQ-F09	Allow authenticated users to upload PDF, DOCX and TXT files.	High
REQ-F10	Validate type, MIME, size and basic integrity.	High
REQ-F11	Assign unique document ID and owner ID.	High
REQ-F12	Expose queued, processing, indexed, failed and deleted states.	High
REQ-F13	Store filename, type, size, checksum and page/section metadata where available.	High
REQ-F14	Allow users to view, rename and delete their own documents.	Medium
REQ-F15	Prevent cross-user document/chunk access.	High

4.3. Document Parsing and Chunking

Priority: High. Ingestion converts source files into retrievable evidence units while preserving provenance.

ID	Requirement	Priority
REQ-F16	Extract machine-readable text from supported documents.	High
REQ-F17	Normalize unnecessary whitespace while preserving meaningful structure.	High
REQ-F18	Split text using configurable chunk size and overlap.	High
REQ-F19	Preserve page, heading/section and chunk-order metadata where available.	High
REQ-F20	Assign stable chunk ID and document/owner association.	High
REQ-F21	Produce an actionable failure state for ingestion errors.	Medium

4.4. Embedding and Vector Storage

Priority: High. Dense retrieval requires consistent embeddings and metadata-aware vector storage.

ID	Requirement	Priority
REQ-F22	Generate dense BGE embeddings for indexed chunks.	High
REQ-F23	Store vectors in Qdrant with ownership and provenance metadata.	High
REQ-F24	Validate vector dimensionality before insertion.	High
REQ-F25	Support authorization-aware metadata filters for dense search.	High
REQ-F26	Support re-indexing when chunk/embedding configuration changes.	Medium

5.1. BM25, Dense and Hybrid Retrieval

Priority: High. These fixed modes form the baseline and the input space of the adaptive router.

ID	Requirement	Priority
REQ-F27	Maintain a BM25-compatible lexical index over normalized chunks.	High
REQ-F28	Return top-K BM25 candidates with scores and provenance.	High
REQ-F29	Retrieve dense candidates from Qdrant using configured similarity.	High
REQ-F30	Return dense scores and provenance.	High
REQ-F31	Combine BM25 and dense candidates using a documented fusion method.	High
REQ-F32	Deduplicate hybrid candidates.	High
REQ-F33	Record retrieval mode and parameters for every run.	High

5.2. Adaptive Retrieval Selection

Priority: High; core research contribution. The router selects BM25, Dense or Hybrid using documented query features, diagnostics or a learned/rule-based policy.

ID	Requirement	Priority
REQ-F34	Analyze each incoming query before final retrieval selection.	High
REQ-F35	Support BM25, Dense and Hybrid choices.	High
REQ-F36	Record selected strategy and decision features/reason.	High
REQ-F37	Provide deterministic fallback when the router cannot decide.	High
REQ-F38	Never bypass authorization filters during routing/retrieval.	High
REQ-F39	Compare adaptive retrieval with fixed retrieval baselines.	High

5.3. Cross-Encoder Reranking and Query Understanding

Priority: High. Reranking refines first-stage candidate ordering.

ID	Requirement	Priority
REQ-F40	Accept configurable top-N candidate set.	High
REQ-F41	Score query–chunk pairs using the Cross-Encoder.	High
REQ-F42	Return top-M reranked chunks with scores/provenance.	High
REQ-F43	Record reranking latency.	Medium
REQ-F44	Classify QA, summarization, comparison, citation/evidence or report tasks.	High
REQ-F45	Identify document scope applicable to the query.	High
REQ-F46	Detect multi-step tasks when supported.	Medium

6.1. LangGraph Multi-Agent Orchestration

Priority: High. LangGraph provides bounded stateful orchestration rather than unrestricted autonomous loops.

ID	Requirement	Priority
REQ-F47	Implement a supervisor/orchestrator responsibility for routing and workflow control.	High
REQ-F48	Implement retrieval/search responsibility for evidence acquisition.	High
REQ-F49	Implement verification responsibility for claim/evidence checking.	High
REQ-F50	Implement synthesis responsibility for final answer generation.	High
REQ-F51	Maintain typed/shared workflow state.	High
REQ-F52	Enforce a maximum number of graph/retrieval/verification iterations.	High
REQ-F53	Terminate with an explicit insufficient-evidence/failure state when appropriate.	High

6.2. Question Answering, Summarization and Comparison

QA is High priority; comparison is Medium priority.

ID	Requirement	Priority
REQ-F54	Answer questions using evidence from the authorized document scope.	High
REQ-F55	Provide source references for document-derived claims.	High
REQ-F56	State when evidence is insufficient rather than inventing document facts.	High
REQ-F57	Generate summaries for selected document(s).	High
REQ-F58	Support configurable summary style/length where implemented.	Medium
REQ-F59	Allow selection of at least two documents for comparison.	Medium
REQ-F60	Identify similarities/differences using retrieved evidence.	Medium
REQ-F61	Include contributing source references for comparison claims where feasible.	Medium

6.3. Evidence Verification

Priority: High; core trustworthiness component. Important draft claims are checked against retrieved evidence. The implementation may use NLI, semantic similarity, an evidence-constrained judge, or a documented hybrid method.

ID	Requirement	Priority
REQ-F62	Identify factual claims requiring document support.	High
REQ-F63	Compare claims with retrieved evidence passages.	High
REQ-F64	Classify evidence as Supported, Partially Supported, Unsupported or Contradicted, or an equivalent documented taxonomy.	High
REQ-F65	Retain evidence passages associated with verification decisions.	High
REQ-F66	Trigger corrective retrieval/revision when important claims are unsupported and budget permits.	High
REQ-F67	Never label an unsupported claim as verified.	High

6.4. Confidence-Aware Explainability

Priority: High. Explainability includes provenance and evidence, not merely a confidence number.

ID	Requirement	Priority
REQ-F68	Display a confidence indicator for the final response.	High
REQ-F69	Display verification status for evidence-backed claims.	High
REQ-F70	Display document name and page/section information when available.	High
REQ-F71	Display supporting evidence passages in expandable/inspectable form.	High
REQ-F72	Expose selected retrieval strategy in a research/explainability view.	Medium
REQ-F73	Explain that confidence is a signal, not a guarantee of correctness.	High

7. Data Requirements and API Requirements

7.1 Core Data Entities

Entity	Key Fields	Purpose
User	id, email, password_hash, role, created_at	Identity and authorization
Document	id, user_id, filename, type, size, status, checksum	Document lifecycle
DocumentChunk	id, document_id, user_id, text, page, section, chunk_index	Retrievable evidence
EmbeddingRecord	chunk_id, model, dimension, vector_ref	Embedding provenance
Query	id, user_id, scope, text, task_type, created_at	User request
RetrievalRun	id, query_id, mode, parameters, candidates, latency	Retrieval diagnostics
AgentRun	id, query_id, graph_version, status, latency	Workflow trace
Claim	id, query_id, text, verification_status, confidence	Claim verification
EvidenceLink	claim_id, chunk_id, score, relation	Claim-to-evidence mapping
Conversation	id, user_id, title, created_at	Chat grouping
EvaluationRun	id, dataset, system_version, config	Experiment record
MetricResult	run_id, metric_name, value, slice	Metric outputs
AuditEvent	id, actor_id, action, resource, timestamp	Governance/audit

7.2 Data Integrity Requirements

- REQ-D01 — Every document shall have an owner/account identifier.
- REQ-D02 — Every chunk shall retain document, owner and provenance identifiers.
- REQ-D03 — Deleted documents shall not remain retrievable from active indexes.
- REQ-D04 — Retrieval and agent logs shall reference the query/configuration used.
- REQ-D05 — Evaluation records shall contain enough metadata to reproduce the run.
- REQ-D06 — Timestamps shall follow a documented consistent timezone strategy.

7.3 Representative REST API Groups

Group	Representative Operations	Access
/auth	register, login, refresh, logout, reset	Public entry points; protected otherwise
/documents	upload, list, get, delete, reprocess	Authenticated + owner scoped
/queries	ask, history, result	Authenticated + owner scoped
/retrieval	inspect candidates/diagnostics	Authenticated/research policy
/conversations	create, list, messages	Authenticated + owner scoped
/evaluation	create, run, results, export	Researcher/Admin
/admin	users, jobs, health, audit	Admin only
/health	liveness/readiness	Deployment policy

7.4 API Behavior

- Return structured errors with code, message and optional correlation ID.
- Use 4xx for validation/authentication failures and safe 5xx responses without stack traces.
- Handle LLM timeouts/rate limits with bounded retries and clear status.
- Expose long-running document/evaluation jobs through status APIs.
- Enforce ownership server-side for every owner-scoped resource.

8. System Architecture and Component Requirements

8.1 Logical Architecture

The proposed architecture is layered: user interface → application APIs → document ingestion → retrieval → adaptive routing → reranking → multi-agent orchestration → verification → explainability → final response. PostgreSQL stores transactional metadata, Qdrant stores dense vectors, and BM25 provides lexical retrieval.

Layer	Components	Responsibility
Client	Next.js, TypeScript, responsive UI	Upload, query, evidence and history presentation
API	FastAPI, JWT, RBAC	Validation, authorization and application services
Ingestion	Parser, cleaner, chunker	Source-to-evidence conversion
Retrieval	BM25, Qdrant dense, hybrid	Candidate evidence acquisition
Adaptive Routing	Query analyzer + router	Select BM25/Dense/Hybrid
Reranking	Cross-Encoder	Refine candidate relevance
Agents	Supervisor, Retriever, Verifier, Synthesizer	Bounded task orchestration
Storage	PostgreSQL, Qdrant, BM25 index	Metadata, vectors, lexical index
Evaluation	Baselines, metrics, ablations	Research measurement

8.2 End-to-End Processing Flow

1) Upload document → 2) validate → 3) parse/normalize → 4) chunk with page/section metadata → 5) create BGE embeddings → 6) store in Qdrant + BM25 → 7) submit query → 8) analyze task/query → 9) adaptively select BM25/Dense/Hybrid → 10) retrieve candidates → 11) Cross-Encoder rerank → 12) LangGraph workflow → 13) verify claims → 14) corrective retrieval if needed → 15) synthesize → 16) return answer + citations + evidence + verification + confidence.

8.3 Architecture Requirements

- REQ-AR01 — Separate retrieval, reasoning, verification and presentation layers.
- REQ-AR02 — Retrieval components shall be configurable/replacable without rewriting the UI.
- REQ-AR03 — Embedding, reranker, retrieval and agent configurations shall be versioned for evaluation.
- REQ-AR04 — Core services shall expose health checks.
- REQ-AR05 — The application shall fail gracefully when optional AI services are unavailable.

9. Other Nonfunctional Requirements

9.1 Performance

ID	Requirement	Priority
REQ-NF01	Interactive queries should meet an agreed target latency under defined normal-load conditions.	High
REQ-NF02	Retrieval, reranking, agent and total latency shall be recorded.	High
REQ-NF03	Long-running ingestion shall be asynchronous where necessary.	Medium
REQ-NF04	File size, request rate and concurrency limits shall be configurable.	High
REQ-NF05	Performance reports shall include workload, dataset and deployment configuration.	High

9.2 Security

ID	Requirement	Priority
REQ-S01	Never store plaintext passwords; use secure password hashing.	High
REQ-S02	Protected APIs shall require authentication.	High
REQ-S03	RBAC shall restrict administrator/evaluation operations.	High
REQ-S04	Document retrieval shall enforce server-side ownership filters.	High
REQ-S05	LLM/database secrets shall remain outside source code and client bundles.	High
REQ-S06	Production traffic containing user data shall use HTTPS.	High
REQ-S07	Authentication, upload and AI-intensive endpoints shall be rate-limited.	High
REQ-S08	Uploaded filenames/content paths shall be safely handled.	High
REQ-S09	Document text shall be treated as untrusted input; prompt-injection defenses shall be applied.	High
REQ-S10	Logs shall not contain passwords, tokens or raw secrets.	High
REQ-S11	Sensitive administrative actions shall be auditable.	Medium

9.3 Reliability

- Failed ingestion/AI jobs shall show actionable status.
- External LLM failures shall not corrupt document/index metadata.
- Retry logic shall be bounded.
- Background jobs should be restartable/recoverable.
- Database schema changes shall be migration-managed.

9.4 Usability

The primary query flow shall be understandable without knowledge of RAG. Evidence shall be accessible without exposing raw logs. Processing states and errors shall be clear. The UI shall be responsive on desktop/tablet and cover core mobile workflows.

9.5 Maintainability and Extensibility

Backend modules shall be separated by responsibility. AI components shall use adapters/interfaces so embedding, reranking and LLM providers can change. Configuration shall be environment-driven. Critical authentication, ownership, retrieval and verification behavior shall be tested.

9.6 Scalability

Vector search, relational operations and background processing should be independently scalable. Large collections should use batching rather than loading the entire corpus into memory for each query.

10. Evaluation and Research Requirements

10.1 Evaluation Purpose

The evaluation determines whether adaptive retrieval and evidence verification provide measurable improvements over fixed retrieval baselines. The same question set and document scope shall be used across comparative systems.

Research integrity rule: this SRS specifies metrics and procedures, not performance results. Actual values must be generated from executed experiments.

10.2 Required Baselines

System	Description
Dense RAG	Dense retrieval followed by the common generation pipeline.
BM25 RAG	Lexical retrieval followed by the common generation pipeline.
Hybrid RAG	Fixed combination of BM25 and dense retrieval.
Hybrid + Reranking	Fixed hybrid retrieval followed by Cross-Encoder reranking.
Proposed	Adaptive BM25/Dense/Hybrid + reranking + multi-agent verification + explainability

10.3 Retrieval Metrics

Metric	Purpose
Precision@K	Fraction of top-K retrieved items that are relevant.
Recall@K	Fraction of relevant evidence retrieved within top-K.
MRR	How early the first relevant item appears.
nDCG@K	Ranking quality with graded relevance.

10.4 Generation/RAG Metrics

Metric	Purpose
Faithfulness	Degree to which generated claims are supported by evidence.
Answer Relevancy	Degree to which response addresses the query.
Context Precision	Relevance of retrieved context.
Context Recall	Coverage of information required for the answer.
Hallucination Rate	Frequency of unsupported factual claims under the defined protocol.

10.5 Trust and Explainability Metrics

Metric	Evaluation Goal
Citation Completeness	Fraction of evidence-requiring claims with appropriate source references.
Evidence Coverage	Degree to which important claims link to supporting passages.
Verification Accuracy	Agreement of verification decisions with human/ground-truth labels.
Confidence Calibration	Relationship between confidence and correctness/support level.
Verification Failure Rate	Unsupported claims incorrectly passing as supported.

10.6 Performance Metrics

- End-to-end response latency.
- First-stage retrieval latency.
- Reranking latency.
- Agent workflow latency.
- LLM token usage/cost where available.
- Document ingestion throughput.

10.7 Ablation Studies

Variant	Changed Component	Research Question
Full System	None	Reference configuration
Ablation 1	No Adaptive Retrieval	Does adaptive selection improve quality?
Ablation 2	No Reranking	Does reranking improve evidence ordering?
Ablation 3	No Verification	Does verification reduce unsupported claims?
Ablation 4	No Multi-Agent Layer	Does orchestration improve complex-task performance?
Ablation 5	No Explainability	What quality/overhead changes occur without explainability?

10.8 Evaluation Requirements

ID	Requirement	Priority
REQ-EV01	Run the same query set across all required baselines.	High
REQ-EV02	Save raw predictions, retrieved evidence and metric outputs.	High
REQ-EV03	Record system/retrieval/model versions for every run.	High
REQ-EV04	Export results in CSV and/or JSON.	High
REQ-EV05	Generate plots/tables from recorded results, not hard-coded values.	High
REQ-EV06	Support the defined ablation configurations.	High
REQ-EV07	Record dataset version, seeds and major configuration values where applicable.	Medium

11. Testing and Validation Requirements

11.1 Testing Strategy

Testing shall cover unit, integration, API, security, retrieval, agent workflow, UI and end-to-end behavior. Software correctness tests and research-quality metrics are distinct and both are required.

11.2 Functional Test Areas

Area	Representative Tests
Authentication	Registration, duplicate identity, login, invalid password, expiry, logout
Authorization	Cross-user access denial, admin-only restrictions
Upload	Supported/unsupported files, size limits, malformed files
Ingestion	Parsing, chunking, metadata, indexing, failure/retry
Retrieval	BM25, Dense, Hybrid, filters, top-K, deduplication
Adaptive Router	Routing categories, fallback, decision logging
Reranking	Candidate scores, ordering, timing
Agents	Graph transitions, state integrity, bounded loops
Verification	Supported/partial/unsupported/contradicted cases
Explainability	Citations, evidence, confidence and status UI
Evaluation	Baselines, exports, repeatability, metrics

11.3 Acceptance Criteria

ID	Acceptance Criterion
AC-01	A new user can register and authenticate.
AC-02	An authenticated user can upload a supported document and see processing status.
AC-03	The corpus can be queried using BM25, Dense and Hybrid retrieval.
AC-04	Adaptive routing selects and records a retrieval mode.
AC-05	Candidates are reranked before final context construction.
AC-06	Normal questions return answers with source references and evidence.
AC-07	Unsupported questions do not receive fabricated document-grounded answers.
AC-08	Verification reports evidence status for important claims.
AC-09	Users cannot access another user's documents/evidence.
AC-10	Evaluation executes required baselines and the proposed system on the same query set.
AC-11	Results are exportable with configuration metadata.
AC-12	The complete system builds and runs using documented deployment steps.

11.4 Negative and Adversarial Tests

- Prompt injection embedded inside a document.
- Queries outside the user's document scope.
- Conflicting statements across documents.
- Question whose answer is absent from the corpus.
- Short keyword vs. long semantic query.
- Identifiers/names/numbers requiring exact matching.
- Malformed or empty documents.
- LLM timeout/rate limit.
- Temporary Qdrant/PostgreSQL unavailability.

12. Business Rules and Operational Policies

Rule	Policy
BR-01	Users may access only authorized documents and conversations.
BR-02	Administrators may access operational metadata required for governance, subject to privacy policy.
BR-03	Uploaded document evidence is the source of truth for document-grounded claims; the LLM is not itself a source.
BR-04	Unsupported claims shall be marked or omitted rather than presented as verified.
BR-05	Adaptive routing shall never sacrifice authorization for retrieval quality.
BR-06	Research results shall be traceable to recorded configurations and evaluation runs.
BR-07	Confidence values are signals, not correctness guarantees.
BR-08	Deleted documents shall be removed or invalidated from active retrieval indexes.
BR-09	Research metrics shall come from actual runs; invented performance values are prohibited.
BR-10	Agent loops shall be bounded by step/time budgets.

12.1 Privacy and Data Handling

- Communicate document storage/retention behavior to users.
- Only required metadata should be stored in logs.
- If external LLMs receive document text, deployment documentation should disclose this and use approved provider data controls.
- Sensitive evaluation data should be anonymized or replaced with public/synthetic data where appropriate.

13. Deployment and Infrastructure Requirements

13.1 Containerized Deployment

Docker Compose is the preferred local/development and demonstration deployment. Individual services may later be moved to managed cloud infrastructure.

Service	Role
frontend	Next.js production web application
backend	FastAPI application
postgres	Transactional database
qdrant	Vector database
redis	Optional cache/queue/rate-limit support
worker	Optional asynchronous ingestion/evaluation worker
reverse proxy	Optional TLS termination/routing

13.2 Configuration

- Database credentials/URLs via environment or secrets.
- Qdrant configuration via environment.
- LLM provider/model/key via environment/secrets.
- Embedding model, chunk size/overlap, top-K, fusion and reranking parameters configurable.
- JWT secrets and token policies configurable.
- CORS origins explicitly configured.

13.3 Health and Observability

- Backend liveness/readiness checks.
- Correlation/request IDs for major workflows.
- Document job status and error visibility.
- Workflow version and timing logs without leaking private source text.

13.4 Backup and Recovery

- PostgreSQL backups according to deployment importance.
- Qdrant snapshots/exports when required.
- Documented procedure to rebuild BM25/Qdrant indexes from source documents.

14. Future Enhancements and To-Be-Determined Items

TBD	Item	Status
TBD-01	OCR for scanned PDFs/images	Future enhancement
TBD-02	Multimodal table/image understanding	Future research extension
TBD-03	Knowledge-graph-assisted retrieval	Optional future extension
TBD-04	Team/shared workspaces	Future collaboration feature
TBD-05	Learned routing policy trained from evaluation data	Future optimization
TBD-06	Streaming token-level generation	Optional UX enhancement
TBD-07	Cloud-drive/enterprise connectors	Future integration
TBD-08	Human reviewer feedback loop for verification	Potential research extension

Future items are not mandatory acceptance requirements unless formally moved into project scope.

Appendix A. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation; retrieve contextual evidence before generation.
BM25	Lexical ranking function for term-based information retrieval.
Dense Retrieval	Semantic retrieval using embedding vectors.
Hybrid Retrieval	Combination of lexical and dense retrieval.
Adaptive Retrieval	Dynamic selection of retrieval strategy based on query characteristics/policy.
Reranking	Second-stage ranking of retrieved candidates with a stronger relevance model.
Cross-Encoder	Model that jointly encodes query and candidate text to estimate relevance.
Qdrant	Vector database used for dense embedding storage and search.
Embedding	Numerical representation of text for semantic similarity.
Chunk	Retrievable segment of a source document.
Provenance	Metadata identifying evidence origin such as document/page/section.
LangGraph	Graph-based orchestration framework for stateful LLM/agent workflows.
Evidence Verification	Checking generated claims against source evidence.
Faithfulness	Degree to which generated claims are supported by evidence.
Precision@K	Fraction of top-K retrieved items that are relevant.
Recall@K	Fraction of relevant items retrieved within top-K.
MRR	Mean Reciprocal Rank.
nDCG	Normalized Discounted Cumulative Gain.
RBAC	Role-Based Access Control.
JWT	JSON Web Token.
NLI	Natural Language Inference used for entailment/contradiction/neutrality analysis.
Hallucination	Unsupported or contradicted generated factual claim under the evaluation protocol.

Appendix B. Analysis Models

B.1 Primary Use Cases

Actor	Use Case	Outcome
User	Register / Login	Authenticated workspace
User	Upload Document	Document enters processing pipeline
User	Ask Question	Verified/explainable answer
User	Summarize	Grounded summary
User	Compare Documents	Evidence-backed comparison
User	Inspect Evidence	Sources and verification status
Researcher/Admin	Run Evaluation	Saved metrics and experiment record
Admin	Manage/Monitor	Operational control

B.2 Question Answering Sequence

User → Next.js: query + scope → FastAPI: authenticate/validate → Query Analyzer: task/features → Adaptive Router: BM25/Dense/Hybrid → Retrieval → Cross-Encoder → LangGraph → Verifier: claims ↔ evidence → Synthesizer → FastAPI → Next.js: answer + citations + passages + confidence.

B.3 Document Ingestion Sequence

User → API: upload → validation → document record → queue/worker → parser → chunker → embedding → Qdrant → BM25 index → status=INDEXED → UI refresh.

B.4 Document State Model

State	Meaning	Next States
UPLOADED	Accepted, processing not started	QUEUED / FAILED
QUEUED	Waiting for worker	PROCESSING / FAILED
PROCESSING	Parsing/indexing	INDEXED / FAILED
INDEXED	Available for retrieval	PROCESSING / DELETED
FAILED	Processing failed	QUEUED / DELETED
DELETED	Inactive	Terminal

Appendix C. Requirement Traceability Matrix

Objective	Requirements	Validation
Secure workspace	REQ-F01–F08, REQ-S01–S11	API, security and UI tests
Document ingestion	REQ-F09–F26	Integration tests with document fixtures
Multiple retrieval modes	REQ-F27–F33	Retrieval tests + metrics
Adaptive retrieval	REQ-F34–F39	Routing tests + baseline comparison
Reranking	REQ-F40–F43	Reranking tests + ranking metrics
Multi-agent analysis	REQ-F47–F61	Graph/state tests + task evaluation
Evidence verification	REQ-F62–F67	Claim/evidence evaluation
Explainability	REQ-F68–F73	UI + citation/evidence metrics
Reproducibility	REQ-EV01–EV07	Experiment logs + exports
Operational quality	REQ-NFxx, REQ-ARxx	Load, reliability, deployment tests

Appendix D. End-to-End Implementation Checklist

Phase	Completion Item	Evidence
1	Repository/environment setup	README + .env.example + build
2	Authentication/RBAC	Passing auth/authorization tests
3	PostgreSQL schema/migrations	Migrations + startup
4	Document upload	Working UI/API
5	PDF/DOCX/TXT parsing	Parser tests
6	Chunking/provenance	Stored chunks with metadata
7	BGE embeddings	Embedding service + dimension checks
8	Qdrant indexing/search	Vector collection + tests
9	BM25 indexing/search	Lexical tests
10	Hybrid retrieval	Fusion tests
11	Cross-Encoder reranking	Ranked candidates + latency
12	Adaptive router	Decision logic + logs
13	LangGraph workflow	Bounded state graph
14	Evidence verification	Claim/evidence status
15	Explainability UI	Citations + passages + confidence
16	Summary/comparison	Task-specific outputs
17	Evaluation pipeline	Baselines + metrics + CSV/JSON
18	Ablation experiments	Defined variants
19	Security hardening	Ownership, injection, secrets, rate limits
20	Testing	Unit/integration/E2E/regression
21	Docker deployment	Compose + health checks
22	Research documentation	Experiment configurations
23	Paper integration	Only actual measured results

Appendix E. Recommended Research Dataset Schema

Field	Description	Type
question_id	Unique evaluation query ID	String
question	Natural-language question	String
document_ids	Allowed source documents	List[String]
gold_chunk_ids	Relevant evidence IDs, if annotated	List[String]
reference_answer	Expected answer/rubric if available	String
task_type	QA / summary / comparison / citation	Enum
difficulty	Optional difficulty label	Categorical
notes	Evaluation notes/edge case labels	String

E.1 Experimental Protocol

- Freeze corpus and evaluation questions before comparative runs.
- Use identical questions and document scope across baselines.
- Keep generation model/configuration constant when studying retrieval changes unless model variation is the variable.
- Record K, reranking M, chunk parameters, embedding/reranker/LLM versions, temperature and system version.
- For subjective metrics, document human-rater or judge-model protocol.
- Report failures and missing answers rather than silently excluding them.
- Export raw outputs and metric summaries so paper tables can be regenerated.

END OF SOFTWARE REQUIREMENTS SPECIFICATION

Adaptive Explainable Multi-Agent Retrieval-Augmented Framework for Intelligent Document
Analysis

This SRS is the implementation and research baseline. Mandatory scope changes should be versioned and approved before becoming part of the baseline.